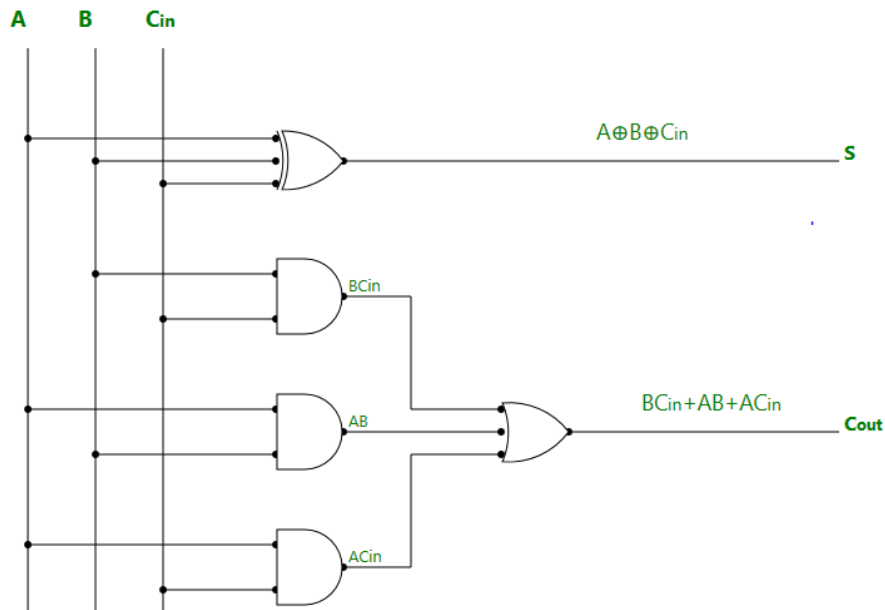


Design of Adder And Subtractor

Adder:

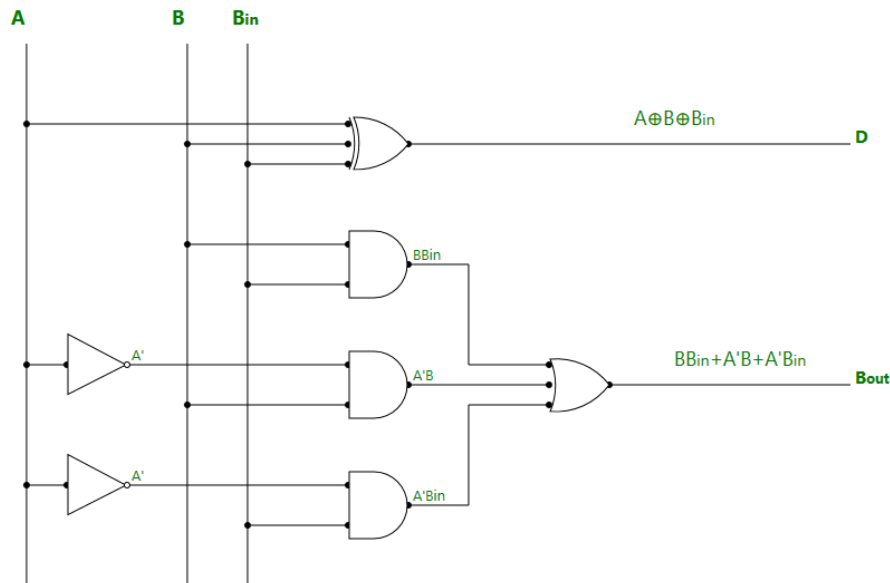


Varilog Module Code:

```
module full_Adder(input a, b, cin, output sum, cout);  
    assign sum = a ^ b ^ cin;  
    assign cout = (a & b) | (b & cin) | (a & cin);  
endmodule
```

Varilog Text Fixture Code:

```
initial begin  
    $monitor("a=%b b=%b cin=%b sum=%b cout=%b", a, b, cin, sum, cout);  
  
    a = 0; b = 0; cin = 0; #10;  
    a = 0; b = 0; cin = 1; #10;  
    a = 0; b = 1; cin = 0; #10;  
    a = 0; b = 1; cin = 1; #10;  
    a = 1; b = 0; cin = 0; #10;  
    a = 1; b = 0; cin = 1; #10;  
    a = 1; b = 1; cin = 0; #10;  
    a = 1; b = 1; cin = 1; #10;  
  
    $finish;  
  
end  
  
endmodule
```



Subtractor:

Varilog Module Code:

```
module subtractor(
    input a, b, bin, output d, bout);
    assign d = a ^ b ^ bin;
    assign bout = (~a & b) | (b & bin) | (~a & bin);
endmodule
```

Varilog Text Fixture Code:

```
initial begin
    $monitor("a=%b b=%b bin=%b d=%b bout=%b", a, b, bin, d, bout);

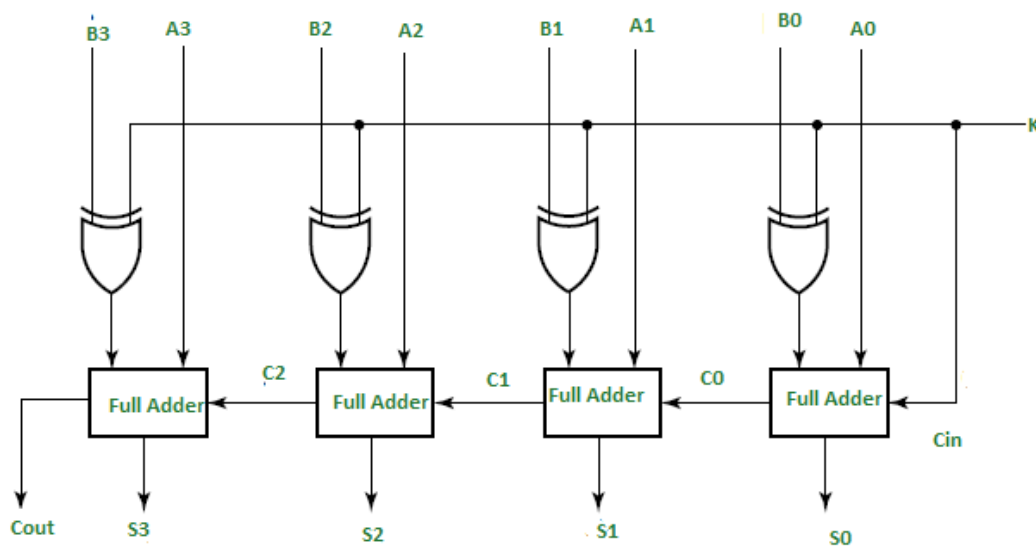
    a = 0; b = 0; bin = 0; #10;
    a = 0; b = 0; bin = 1; #10;
    a = 0; b = 1; bin = 0; #10;
    a = 0; b = 1; bin = 1; #10;
    a = 1; b = 0; bin = 0; #10;
    a = 1; b = 0; bin = 1; #10;
    a = 1; b = 1; bin = 0; #10;
    a = 1; b = 1; bin = 1; #10;

    $finish;

end
```

endmodule

Design of 4-Bit Adder and Subtractor



Varilog Module Code:

```
module addsub(a,b,cin,s,cout);  
input [3:0] a,b;  
input cin;
```

```
output [3:0]s;  
output cout;  
wire [3:0]t;
```

```
assign t = b^{4{cin}};  
assign {cout,s} = a+t+cin;
```

endmodule

Varilog Text Fixture Code:

```
initial begin
```

```
    a = 5; b = 5; cin = 0; #100;  
    a = 3; b = 2; cin = 0; #100;  
    a = 5; b = 3; cin = 0; #100;  
    a = 5; b = 5; cin = 1; #100;  
    a = 3; b = 2; cin = 1; #100;  
    a = 5; b = 3; cin = 1; #100;
```

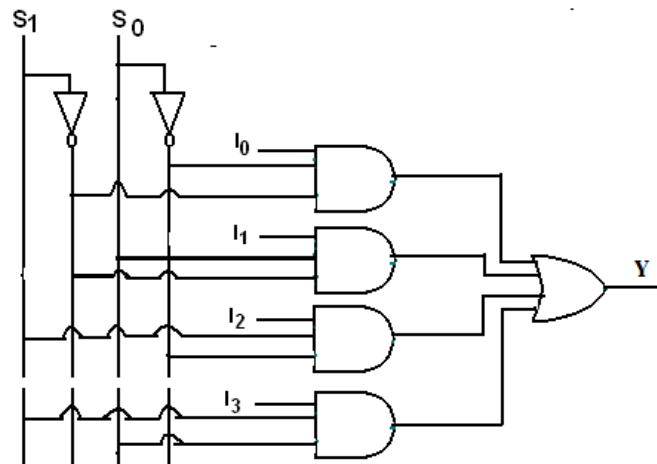
```
end
```

endmodule

Design and Implementation of 4:1 Multiplexer and 1:4 Demultiplexer

Input	S1	S0	Y
I ₀	0	0	I ₀
I ₁	0	1	I ₁
I ₂	1	0	I ₂
I ₃	1	1	I ₃

$$Y = S_1 S_0 I_3 + S_1 \bar{S}_0 I_2 + \bar{S}_1 S_0 I_1 + \bar{S}_1 \bar{S}_0 I_0$$



4 to 1 Multiplexer and its truth table

Varilog Module Code:

```
`timescale 1ns / 1ps
////////////////////////////////////////////////////////////////
// Company:
// Engineer:
//
// Create Date: 18:35:05 08/22/2023
// Design Name:
// Module Name: mult
// Project Name:
// Target Devices:
// Tool versions:
// Description:
//
// Dependencies:
//
// Revision:
// Revision 0.01 - File Created
// Additional Comments:
//
////////////////////////////////////////////////////////////////
module mult(

    input wire [3:0] data_in, // 4 data inputs
    input wire [1:0] select, // 2-bit select input
```

```

        output wire y          // MUX output
    );

    assign y = (select == 2'b00) ? data_in[0] :
        (select == 2'b01) ? data_in[1] :
        (select == 2'b10) ? data_in[2] :
        (select == 2'b11) ? data_in[3] :
        1'b0; // Default output when select is invalid (optional);

endmodule

```

Varilog text fixer:

```

module dsgsdgasd;

    // Inputs
    reg [3:0] data_in;
    reg [1:0] select;

    // Outputs
    wire y;

    // Instantiate the Unit Under Test (UUT)
    mult uut (
        .data_in(data_in),
        .select(select),
        .y(y)
    );

    initial begin
        // Initialize Inputs
        data_in = 0;
        select = 0;

        // Wait 100 ns for global reset to finish
        #100;

        // Add stimulus here
        // Test Case 1
        data_in = 4'b0011;
        select = 2'b10;
        #50;
    end
endmodule

```

```
// Test Case 2
data_in = 4'b1100;
select = 2'b01;
#50;

// Test Case 3
data_in = 4'b0101;
select = 2'b10;
#50;

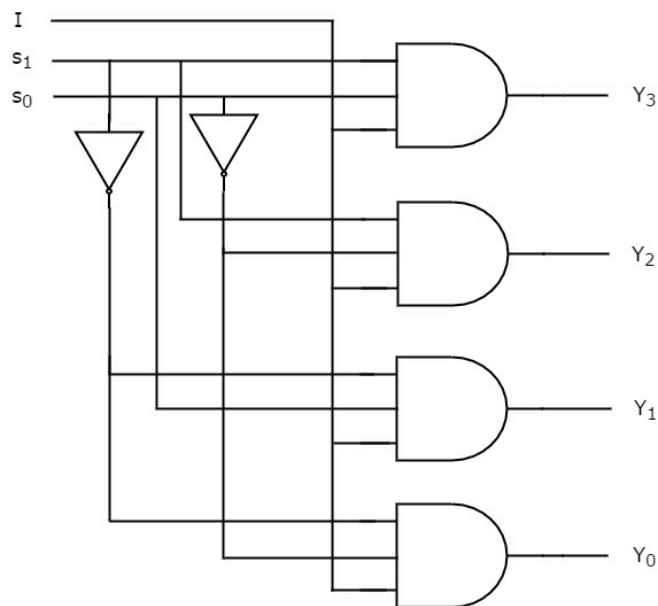
// Test Case 4
data_in = 4'b1010;
select = 2'b00;
#50;

// Test Case 5
data_in = 4'b1111;
select = 2'b11;
#50;

// Add more test cases here

// End simulation
$finish;
end

endmodule
```



Selection Inputs		Outputs			
S ₁	S ₀	Y ₃	Y ₂	Y ₁	Y ₀
0	0	0	0	0	I
0	1	0	0	I	0
1	0	0	I	0	0
1	1	I	0	0	0

From the above Truth table, we can directly write the **Boolean functions** for each output as

$$Y_3 = s_1 s_0 I$$

$$Y_2 = s_1 s_0' I$$

$$Y_1 = s_1' s_0 I$$

$$Y_0 = s_1' s_0' I$$

Varilog Module Code:

```
module demux(
    input wire [1:0] select, // 2-bit select input
    input wire d_in,        // Data input
    output wire [3:0] y      // 4 demux outputs
);
```

```
assign y[0] = (select == 2'b00) ? d_in : 1'b0;
assign y[1] = (select == 2'b01) ? d_in : 1'b0;
assign y[2] = (select == 2'b10) ? d_in : 1'b0;
assign y[3] = (select == 2'b11) ? d_in : 1'b0;

endmodule
```

Varilog text fixer:

```
module test;

    // Inputs
    reg [1:0] select;
    reg d_in;

    // Outputs
    wire [3:0] y;

    // Instantiate the Demultiplexer (DEMUX)
    demults uut (
        .select(select),
        .d_in(d_in),
        .y(y)
    );

    initial begin
        // Initialize Inputs
        select = 2'b00;
        d_in = 1'b1;

        // Wait 100 ns for initialization
        #100;

        // Add stimulus here
        // Test Case 1
        select = 2'b10;
        d_in = 1'b1;
        #50;

        // Test Case 2
        select = 2'b01;
```



```

// Engineer:
//
// Create Date: 23:14:57 10/08/2023
// Design Name: BCDtoEXCESS3
// Module Name: C:/Users/ASUS/Desktop/New folder/helloalsa/test.v
// Project Name: helloalsa
// Target Device:
// Tool versions:
// Description:
//
// Verilog Test Fixture created by ISE for module: BCDtoEXCESS3
//
// Dependencies:
//
// Revision:
// Revision 0.01 - File Created
// Additional Comments:
//
////////////////////////////////////

```

```

module test;

    // Inputs
    reg a;
    reg b;
    reg c;
    reg d;

    // Outputs
    wire [3:0] wxyz;

    // Instantiate the Unit Under Test (UUT)
    BCDtoEXCESS3 uut (
        .a(a),
        .b(b),
        .c(c),
        .d(d),
        .wxyz(wxyz)
    );

    initial begin
        {a,b,c,d} = 4'b0000;
        #100;
        {a,b,c,d} = 4'b0001;
    end

```

```

#100;
{a,b,c,d} = 4'b0010;
#100;
{a,b,c,d} = 4'b0011;
#100;
{a,b,c,d} = 4'b0100;
#100;
{a,b,c,d} = 4'b0101;
#100;
{a,b,c,d} = 4'b0110;
#100;
{a,b,c,d} = 4'b0111;
#100;
{a,b,c,d} = 4'b1001;
#100;
{a,b,c,d} = 4'b1010;
#100;
{a,b,c,d} = 4'b1011;
#100;
{a,b,c,d} = 4'b1100;
#100;
{a,b,c,d} = 4'b1101;
#100;
{a,b,c,d} = 4'b1110;
#100;
{a,b,c,d} = 4'b1111;
#100;

```

end

endmodule

Excess-3 to bcd:

Verilog module:

```

module ex3_to_bcd(
    input w,
    input x,
    input y,
    input z,
    output a,
    output b,
    output c,
    output d
);
assign a = ((w & x) | (w & y & z));

```

```
assign b = (((~x) & (~y)) | ((~x) & (~z)) | (x & y & z));
assign c = (((~y) & z) | (y & (~z)));
assign d = ~z;
```

```
endmodule
```

Text fixture:

```
module test;
```

```
    // Inputs
```

```
    reg w;
```

```
    reg x;
```

```
    reg y;
```

```
    reg z;
```

```
    // Outputs
```

```
    wire a;
```

```
    wire b;
```

```
    wire c;
```

```
    wire d;
```

```
    // Instantiate the Unit Under Test (UUT)
```

```
    ex3_to_bcd uut (
```

```
        .w(w),
```

```
        .x(x),
```

```
        .y(y),
```

```
        .z(z),
```

```
        .a(a),
```

```
        .b(b),
```

```
        .c(c),
```

```
        .d(d)
```

```
    );
```

```
    initial begin
```

```
        // test case -1
```

```
        w = 0;
```

```
        x = 0;
```

```
        y = 1;
```

```
        z = 1;
```

```
        // Wait 100 ns for global reset to finish
```

```
        #100;
```

```
        // test case -1
```

```
w = 0;
x = 1;
y = 0;
z = 0;

// Wait 100 ns for global reset to finish
#100;

// test case -2
w = 0;
x = 1;
y = 0;
z = 1;

// Wait 100 ns for global reset to finish
#100;

// test case -3
w = 0;
x = 1;
y = 1;
z = 0;

// Wait 100 ns for global reset to finish
#100;

// test case -4
w = 0;
x = 1;
y = 1;
z = 1;

// Wait 100 ns for global reset to finish
#100;

// test case -5
w = 1;
x = 0;
y = 0;
z = 0;

// Wait 100 ns for global reset to finish
#100;
```

```
// test case -6
w = 1;
x = 0;
y = 0;
z = 1;

// Wait 100 ns for global reset to finish
#100;

// test case -7
w = 1;
x = 0;
y = 1;
z = 0;

// Wait 100 ns for global reset to finish
#100;

// test case -8
w = 1;
x = 0;
y = 1;
z = 1;

// Wait 100 ns for global reset to finish
#100;

// test case -9
w = 1;
x = 1;
y = 0;
z = 0;

// Wait 100 ns for global reset to finish
#100;

// Add stimulus here
```

```
end
```

```
endmodule
```

MOD-10:

Module:

```
module mod10(
    output reg [3:0] out, // Output of the counter
    input clk,           // Clock input
    input reset          // Reset input (active high)
);

    always @(posedge clk) begin
        if (reset) begin
            out <= 4'b0;
        end else if (out < 9) begin
            out <= out + 1;
        end else begin
            out <= 4'b0;
        end
    end
endmodule
```

Text fixer:

```
`timescale 1ns / 1ps
```

```
////////////////////////////////////////////////////////////////
```

```
// Company:
```

```
// Engineer:
```

```
//
```

```
// Create Date: 00:23:50 10/09/2023
```

```
// Design Name: mod10
```

```
// Module Name: C:/Users/ASUS/Desktop/New folder/sgasdgasgdasdg/test.v
```

```
// Project Name: sgasdgasgdasdg
```

```
// Target Device:
```

```
// Tool versions:
```

```
// Description:
```

```
//
```

```
// Verilog Test Fixture created by ISE for module: mod10
```

```
//
```

```
// Dependencies:
```

```
//
```

```
// Revision:
```

```
// Revision 0.01 - File Created
```

```
// Additional Comments:
```

```
//
```

//

module test;

 // Inputs
 reg clk;
 reg reset;

 // Outputs
 wire [3:0] out;

 // Instantiate the Unit Under Test (UUT)
 mod10 uut (
 .out(out),
 .clk(clk),
 .reset(reset)
);

 initial begin
 clk = 1'b1;
 forever #5 clk = ~clk;
 end

 initial begin
 reset = 1;
 #10 reset = 0;
 end

endmodule

```

#include<bits/stdc++.h>
using namespace std;
class Node
{
public:
    int val;
    Node *next;
    Node(int val)
    {
        this->val=val;
        this->next=NULL;
    }
};
void listInsert(Node *&head, int pos,int val)
{
    if(pos==0)
    {
        Node *prev_next = head;
        head = new Node(val);
        head->next=prev_next;
        return ;
    }
    pos--;
    Node * temp = head;
    for(int i = 0; i<pos; i++)
    {
        temp = temp->next;
    }
    Node *ins = new Node(val);
    Node *save_next = temp->next;
    temp->next = ins;
    ins->next= save_next;
}
void deletNode(Node *&head, int pos)
{
    if(pos==0)
    {
        head = head -> next;
        return ;
    }
    pos--;
    Node *temp = head;
    for(int i = 0; i<pos; i++)
    {

```

```

        temp=temp->next;
    }
    Node *c = temp -> next -> next;
    delete temp->next;
    temp->next=c;

}
void printList(Node *head)
{
    Node *temp = head;
    while(temp != NULL)
    {
        cout<<temp->val<<endl;
        temp = temp -> next;
    }
}
int main()
{
    Node *head = new Node(5);
    Node *a = new Node(3);
    Node *b = new Node (9);
    Node *c = new Node (42);
    Node *d = new Node (0);
    Node *e = new Node (10);
    head->next=a;
    a->next=b;
    b->next=c;
    c->next=d;
    d->next=e;
    listInsert(head,4,500);
    printList(head);
    return 0;
}

```